

Nesne Tabanlı Programlama

static ve this Keyword'leri, Method Overloading, Method Overriding ve super

Java ile Nesne Tabanlı Programlama · Dr. Öğr. Üyesi Baki Gökgöz · Gümüşhane Üniversitesi

Bu Derste Neler Öğreneceğiz?

static Keyword

Class'a ait variable, method, block ve nested class'lar; tüm object'lerin paylaştığı üyeler.

this Keyword

Mevcut object'e referans; constructor chaining, method chaining.

Method Overloading

Aynı isim, farklı parametre: compile-time polymorphism ve type promotion.

Method Overriding

Subclass'ın parent method'unu yeniden tanımlaması: runtime polymorphism.

super Keyword

Parent class'ın field, method ve constructor'ına erişim.

Alıştırmalar

Çıktı tahmini, sık hatalar ve karşılaştırma tabloları.

Bölüm 1

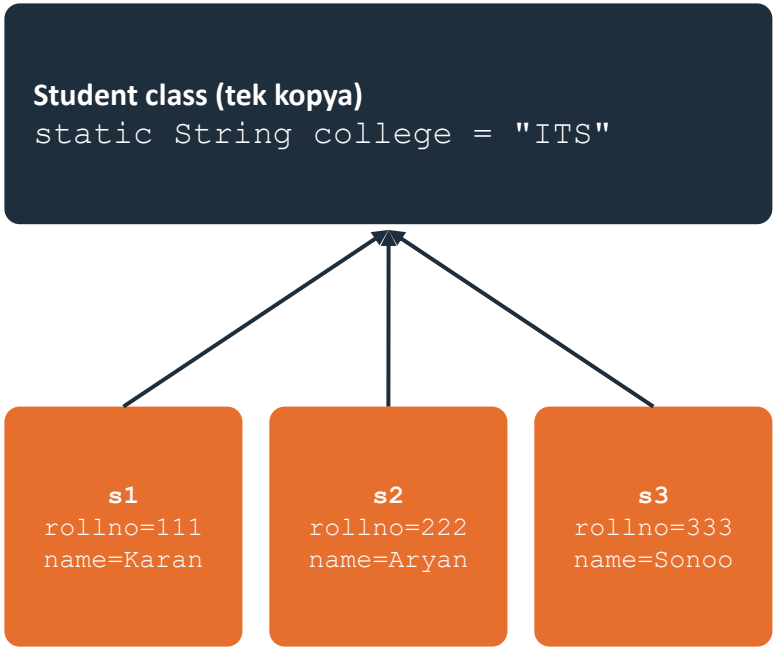


static Keyword

Object'e değil, class'a ait üyeler

static: Class'a Ait, Herkesin Ortak Malı

- `static` üye **object'e değil class'a** aittir; bellekte **tek kopya** vardır.
- Kullanıldığı yerler: **variable** (class variable), **method** (class method), **block**, **nested class**.
- Object oluşturmadan erişilir: `ClassName.member`
- static variable class yüklenirken bir kez oluşturulur ve default değer alır.
- `main` static'tir: JVM programı object oluşturmadan başlatabilsin diye.



Her object kendi instance variable'larına sahip, college'ı ise paylaşıyor.

static Variable

```java
class Student {
    int rollno;                        // instance variable
    String name;
    static String college = "ITS";     // static variable
    Student(int r, String n) { rollno = r; name = n; }
    void display() {
        System.out.println(rollno + " " + name + " " + college);
    }
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(111, "Karan");
        Student s2 = new Student(222, "Aryan");
        s1.display();
        Student.college = "BBDIT";     // herkes için değişir
        s1.display();
        s2.display();
    }
}
```

Çıktı (Output)

```
111 Karan ITS
111 Karan BBDIT
222 Aryan BBDIT
```

- Ortak özellikler için kullanılır: okul adı, şirket adı, faiz oranı.
- Her object için ayrı kopya tutulmadığından **bellek tasarrufu** sağlar.
- Tek satırlık değişiklik (`Student.college = …`) tüm object'leri etkiler.

Sayaç Örneği: Instance mı, static mi?

Instance variable: her object'te ayrı

```
class Counter {
    int count = 0;
    Counter() {
        count++;
        System.out.println(count);
    }
}
new Counter();
new Counter();
new Counter();
```

Çıktı (Output)

```
1
1
1
```

- Her object kendi **`count`**'unu 0'dan başlatır.

static variable: tek ortak kopya

```
class Counter {
    static int count = 0;
    Counter() {
        count++;
        System.out.println(count);
    }
}
new Counter();
new Counter();
new Counter();
```

Çıktı (Output)

```
1
2
3
```

- Tüm object'ler aynı **`count`**'u artırır.

static Method

```java
class Student {
    int rollno;
    String name;
    static String college = "ITS";
    static void change() {              // static method
        college = "BBDIT";
    }
    Student(int r, String n) { rollno = r; name = n; }
    void display() {
        System.out.println(rollno + " " + name + " " + college);
    }
}
class Calculate {
    static int cube(int x) { return x * x * x; }    // utility
}
public class Main {
    public static void main(String[] args) {
        Student.change();               // object gerekmez
        new Student(111, "Karan").display();
        System.out.println(Calculate.cube(5));
    }
}
```

Çıktı (Output)

```
111 Karan BBDIT
125
```

- static method object olmadan, **class adıyla** çağrılır.
- static variable'ları okuyup değiştirebilir.
- Tipik kullanım: **utility** method'lar (**`Math.max`**, **`Integer.parseInt`**).

static Method'un Kısıtlamaları

✗ Instance variable'a doğrudan erişim

```
class A {
    int a = 40;   // non-static
    public static void main(String[] args) {
        System.out.println(a);   // HATA
    }
}
```

Derleme Hatası

```
error: non-static variable a cannot be
referenced from a static context
```

- static method hangi object'in **a**'sını kullanacağını bilemez.
- **this** ve **super** da static context'te kullanılamaz.

✓ Önce object oluştur

```
class A {
    int a = 40;
    public static void main(String[] args) {
        A obj = new A();
        System.out.println(obj.a);
    }
}
```

Çıktı (Output)

```
40
```

- Instance üyelere bir object üzerinden erişilir.

static Block ve Çalışma Sırası

```java
public class Main {
    static int max;

    static {  // static block
        System.out.println("1) static block");
        max = 100;
    }
    {  // instance block
        System.out.println("3) instance block");
    }
    Main() {
        System.out.println("4) constructor");
    }
    public static void main(String[] args) {
        System.out.println("2) main, max = " + max);
        new Main();
    }
}
```

Çıktı (Output)

```
1) static block
2) main, max = 100
3) instance block
4) constructor
```

- **static block**: class belleğe yüklenirken **bir kez**, **`main`**'den önce çalışır; static variable'ları hazırlar.
- **Instance block**: her object oluşturulduğunda constructor'dan önce çalışır.
- JDK 7'den beri **`main`** olmadan sadece static block ile program çalıştırılamaz.

static Nested Class

```java
class Library {
    static int bookCount = 0;

    static class Book {                  // static nested class
        String title;
        Book(String title) {
            this.title = title;
            bookCount++;                 // outer'ın static üyesi
        }
        void info() {
            System.out.println("Kitap: " + title);
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Library.Book b1 = new Library.Book("Java 101");
        Library.Book b2 = new Library.Book("OOP");
        b1.info();
        System.out.println("Toplam: " + Library.bookCount);
    }
}
```

Çıktı (Output)

```
Kitap: Java 101
Toplam: 2
```

- Outer class'ın object'i olmadan oluşturulur: **`new Library.Book(...)`**.
- Outer class'ın yalnızca **static** üyelerine doğrudan erişebilir.
- Birbiriyle ilgili class'ları mantıksal olarak gruplamak için kullanılır.

static ve Instance Üyelerin Karşılaştırması

Özellik	static	Instance
Kime ait?	Class'a	Her bir object'e
Kopya sayısı	Tek (tüm object'ler paylaşır)	Object başına bir
Oluşma zamanı	Class yüklenince	new ile object oluşunca
Erişim	ClassName.member	obj.member
Diğer tür üyeye erişim	Instance üyeye doğrudan erişemez	static üyelere erişebilir
this / super	Kullanılamaz	Kullanılabilir
Overriding	Override edilemez (method hiding)	Override edilebilir
Tipik kullanım	Sabitler, sayaçlar, utility method'lar	Object'e özgü durum ve davranış

Bölüm 2



this Keyword

Mevcut object'in kendisine referans

this Keyword'ün 6 Kullanımı

1. Instance variable

Parametre ile aynı isimdeki field'ı ayırt eder: `this.name = name;`

2. Method çağırma

Aynı class'ın method'unu çağırır: `this.m()` (yazılmasa da compiler ekler).

3. this()

Aynı class'taki başka bir constructor'ı çağırır (**constructor chaining**).

4. Method argümanı

Mevcut object'i başka bir method'a gönderir: `printer.print(this)`.

5. Constructor argümanı

Mevcut object'i başka bir class'ın constructor'ına verir: `new B(this)`.

6. return this

Mevcut object'i döndürür → **method chaining**: `obj.a().b().c()`.

this ile Instance Variable'a Erişim

✗ this yok: parametre kendine atanır

```
class Student {
    int rollno;
    String name;
    Student(int rollno, String name) {
        rollno = rollno;   // parametre!
        name = name;
    }
}
Student s = new Student(111, "Ankit");
System.out.println(s.rollno + " " + s.name);
```

Çıktı (Output)

```
0 null
```

- Local parametre field'ı **gölgeler** (shadowing).
- Field'lar default değerinde kalır.

✓ this.field = parametre

```
class Student {
    int rollno;
    String name;
    Student(int rollno, String name) {
        this.rollno = rollno;
        this.name = name;
    }
}
Student s = new Student(111, "Ankit");
System.out.println(s.rollno + " " + s.name);
```

Çıktı (Output)

```
111 Ankit
```

- `this.rollno` → field, `rollno` → parametre.

this() ile Constructor Chaining

```java
class Student {
    int rollno;
    String name;
    float fee;
    Student(int rollno, String name) {
        this.rollno = rollno;
        this.name = name;
    }
    Student(int rollno, String name, float fee) {
        this(rollno, name);          // ortak kodu tekrar kullan
        this.fee = fee;
    }
    void display() {
        System.out.println(rollno + " " + name + " " + fee);
    }
}
public class Main {
    public static void main(String[] args) {
        new Student(111, "ankit").display();
        new Student(112, "sumit", 6000f).display();
    }
}
```

Çıktı (Output)

```
111 ankit 0.0
112 sumit 6000.0
```

- Ortak başlatma kodu **tek bir** constructor'da tutulur; tekrar önlenir.
- `this(...)` constructor'ın **ilk satırı** olmalıdır.
- Diğer constructor'lar ona zincirlenir (chaining).

Kural: this() İlk Statement Olmalı

✗ this() ikinci satırda

```
class Box {
    int w, h;
    Box(int w) { this.w = w; }
    Box(int w, int h) {
        this.h = h;
        this(w);          // HATA
    }
}
```

Derleme Hatası

```
error: call to this must be first
statement in constructor
```

✓ this() ilk satırda

```
class Box {
    int w, h;
    Box(int w) { this.w = w; }
    Box(int w, int h) {
        this(w);
        this.h = h;
    }
}
Box b = new Box(3, 4);
System.out.println(b.w + "x" + b.h);
```

Çıktı (Output)

```
3x4
```

this ile Mevcut Object'i Argüman Olarak Göndermek

```java
class Printer {
    void print(Book b) {
        System.out.println("Yazdırılıyor: " + b.title);
    }
}
class Book {
    String title;
    Book(String title) { this.title = title; }
    void printWith(Printer p) {
        p.print(this);  // kendini gönder
    }
}
public class Main {
    public static void main(String[] args) {
        Book book = new Book("Java OOP");
        book.printWith(new Printer());
    }
}
```

Çıktı (Output)

```
Yazdırılıyor: Java OOP
```

- **`p.print(this)`**: object, kendisini başka bir object'in method'una verir.
- Aynı şekilde constructor'a da verilebilir: **`new Helper(this)`**.
- Event handling'de (ör. butona dinleyici olarak kendini vermek) çok kullanılır.

return this ile Method Chaining

```java
class Counter {
    int value;
    Counter add(int n) {
        value += n;
        return this;                    // kendini döndür
    }
    Counter reset() {
        value = 0;
        return this;
    }
}
public class Main {
    public static void main(String[] args) {
        Counter c = new Counter().add(5).add(10).add(2);
        System.out.println(c.value);
        System.out.println(c.reset().add(3).value);

        StringBuilder sb = new StringBuilder();
        sb.append("Nesne ").append("Tabanlı ").append("Programlama");
        System.out.println(sb);
    }
}
```

Çıktı (Output)

```
17
3
Nesne Tabanlı Programlama
```

- Method'un return type'ı class'ın kendisi olur ve **`return this;`** yazılır.
- Böylece çağrılar art arda zincirlenir: **fluent API**.
- JDK'dan örnek: **`StringBuilder.append()`**.

this Gerçekten Mevcut Object mi?

```java
public class Main {
    void check(Main other) {
        System.out.println(this == other);
    }
    void hello() {
        System.out.println("hello");
    }
    void run() {
        hello();  // compiler bunu this.hello() yapar
        this.hello();
    }
    public static void main(String[] args) {
        Main obj = new Main();
        obj.check(obj);  // aynı object
        obj.check(new Main());  // farklı object
        obj.run();
    }
}
```

Çıktı (Output)

```
true
false
hello
hello
```

- Method'u çağıran object **`obj`** ise, method içindeki **`this`** de **o object**'tir.
- **`System.out.println(obj)`** ve method içinde **`println(this)`** aynı **`Main@1b6d3586`** benzeri değeri yazar.
- **`this`** static method'larda kullanılamaz.

Bölüm 3

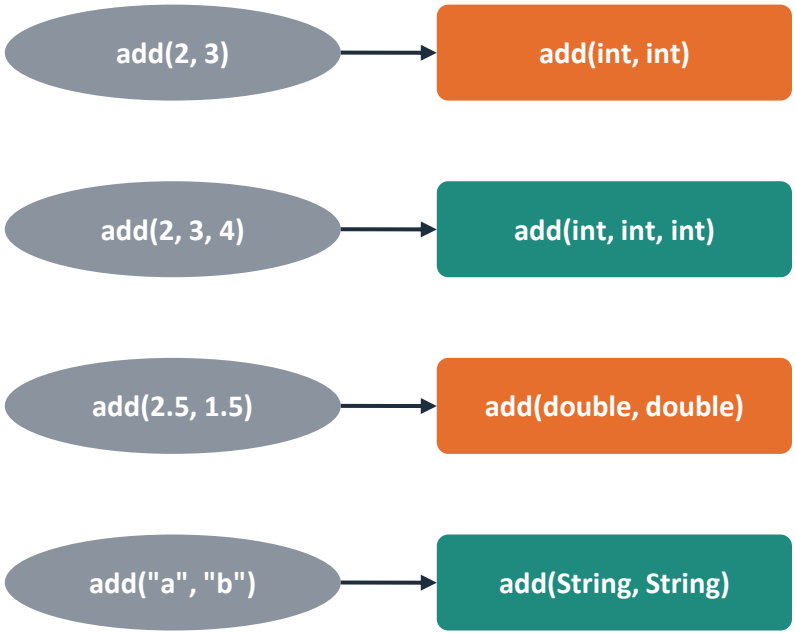


Method Overloading

Aynı isim, farklı parametre listesi

Method Overloading Nedir?

- Aynı class'ta **aynı isimde** ama **farklı parametre listesine** sahip birden fazla method.
- Parametreler şunlarla farklılaşır: **sayı**, **tip** veya **sıra**.
- Sadece **return type**'ı farklı olmak yetmez → compile error.
- Hangi method'un çağrılacağına **compile time**'da karar verilir → **compile-time (static) polymorphism**.
- `a(int,int)` ve `b(int,int,int)` yerine tek isim: `add(...)` → okunabilirlik.



Overloading: Sayı, Tip ve Sıra ile

```java
class Adder {
    static int add(int a, int b) { return a + b; }
    static int add(int a, int b, int c) { return a + b + c; }
    static double add(double a, double b) { return a + b; }
    static String add(String a, String b) { return a + b; }
}
class Printer {
    static void show(String s, int n) { System.out.println("S,I"); }
    static void show(int n, String s) { System.out.println("I,S"); }
}
public class Main {
    public static void main(String[] args) {
        System.out.println(Adder.add(11, 11));
        System.out.println(Adder.add(11, 11, 11));
        System.out.println(Adder.add(12.5, 12.25));
        System.out.println(Adder.add("Java", "!"));
        Printer.show("a", 1);
        Printer.show(1, "a");
    }
}
```

Çıktı (Output)

```
22
33
24.75
Java!
S,I
I,S
```

- Parametre **sayısı** farklı: **`add(int,int)` / `add(int,int,int)`**
- Parametre **tipi** farklı: **`add(int,int)` / `add(double,double)`**
- Parametre **sırası** farklı: **`show(String,int)` / `show(int,String)`**

Sadece Return Type Farkı Overloading Değildir

✗ Aynı parametre, farklı return type

```
class Adder {
    static int add(int a, int b) {
        return a + b;
    }
    static double add(int a, int b) {
        return a + b;
    }
}
```

Derleme Hatası

```
error: method add(int,int) is already
defined in class Adder
```

- **`add(11, 11)`** çağrısında hangisi seçilecek? Compiler bilemez.

✓ JDK'dan örnek: println overloading

```
System.out.println(5);
System.out.println('A');
System.out.println(2.5);
System.out.println(true);
System.out.println("Java");
System.out.println();  // boş satır
```

Çıktı (Output)

```
5
A
2.5
true
Java
```

- **`PrintStream`** class'ında 10 farklı **`println`** vardır.

Overloading ve Type Promotion

```java
class Calc {
    void sum(int a, long b)  { System.out.println("int,long"); }
    void sum(long a, long b) { System.out.println("long,long"); }
    void show(int a)    { System.out.println("int"); }
    void show(double a) { System.out.println("double"); }
}
public class Main {
    public static void main(String[] args) {
        Calc c = new Calc();
        c.sum(20, 20);     // tam eşleşme yok -> promotion
        c.show(5);         // tam eşleşme: int
        c.show(5L);        // long -> double'a yükselir
        c.show('A');       // char -> int'e yükselir
        c.show(2.5f);      // float -> double
    }
}
```

Çıktı (Output)

```
int,long
int
double
int
double
```

- Tam eşleşme yoksa argüman **genişletilir** (byte→short→int→long→float→double, char→int).
- Birden fazla aday varsa **en spesifik** olan seçilir.
- Daraltma (double → int) asla otomatik yapılmaz.

Type Promotion'da Ambiguity

✗ İki aday eşit derecede uygun

```
class Calc {
    void sum(int a, long b) {
        System.out.println("a");
    }
    void sum(long a, int b) {
        System.out.println("b");
    }
}
new Calc().sum(20, 20);    // HATA
```

Derleme Hatası

```
error: reference to sum is ambiguous
```

- İki method da eşit derecede uygun.

✓ Argümanı açıkça belirt

```
class Calc {
    void sum(int a, long b) {
        System.out.println("a");
    }
    void sum(long a, int b) {
        System.out.println("b");
    }
}
new Calc().sum(20, 20L);  // long literal
```

Çıktı (Output)

```
a
```

- `20L` ile ikinci argüman zaten long → ilk method tam eşleşir.

main() Method'u da Overload Edilebilir

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("main with String[]");
        main("test");  // diğerlerini biz çağırırız
        main();
    }
    public static void main(String arg) {
        System.out.println("main with String");
    }
    public static void main() {
        System.out.println("main without args");
    }
}
```

Çıktı (Output)

```
main with String[]
main with String
main without args
```

- JVM yalnızca **`public static void main(String[] args)`** imzasını **giriş noktası** olarak kullanır.
- Diğer **`main`** method'ları sıradan overloaded method'lardır; ancak açıkça çağrılırsa çalışır.

Overloading: Avantajlar ve Dikkat Edilecekler



Avantajlar

- **Okunabilirlik**: benzer işler aynı isimle
- **Esnek API**: farklı tip/sayıda argümanla çağrılabilir (`println`, `Math.max`)
- Compile-time polymorphism sağlar
- **Performans maliyeti yoktur**: seçim derleme zamanında yapılır



Dikkat edilecekler

- Type promotion **ambiguity** hatalarına yol açabilir
- Çok fazla overload kodun anlaşılmasını zorlaştırır
- Overload'lar **aynı anlamda** iş yapmalı; farklı işler için farklı isim kullanın
- Hangi versiyonun çağrıldığı her zaman açık olmayabilir: dokümante edin

Bölüm 4



Method Overriding

Subclass'ın parent davranışını yeniden tanımlaması

Overriding Olmadan ve Overriding ile

Override yok: parent'ın method'u çalışır

```
class Vehicle {
    void run() {
        System.out.println("Vehicle is running");
    }
}
class Bike extends Vehicle { }

new Bike().run();
```

Çıktı (Output)

```
Vehicle is running
```

- Bike, run()'ı olduğu gibi **miras** alır.

Override var: subclass'ın method'u çalışır

```
class Vehicle {
    void run() {
        System.out.println("Vehicle is running");
    }
}
class Bike extends Vehicle {
    @Override
    void run() {
        System.out.println("Bike is running");
    }
}
new Bike().run();
```

Çıktı (Output)

```
Bike is running
```

- Aynı isim + aynı parametre + IS-A ilişkisi = **overriding**.

Method Overriding Kuralları

İmza aynı olmalı

- Aynı method adı
- Aynı parametre sayısı ve tipleri
- **IS-A** (inheritance) ilişkisi gerekli

Return type

- Aynı tip veya alt tipi (**covariant return type**, Java 5+)
- Primitive'lerde birebir aynı olmalı

Access modifier

- Daha **kısıtlayıcı** olamaz
- `public` → yalnızca `public`
- `protected` → `protected`/`public`

Override edilemeyenler

- `final` ve `private` method'lar
- `static` method'lar (hiding olur)
- Constructor'lar

Gerçek Dünya Örneği: Banka Faiz Oranları

```java
class Bank {
    int getRateOfInterest() { return 0; }
}
class SBI extends Bank {
    @Override int getRateOfInterest() { return 8; }
}
class ICICI extends Bank {
    @Override int getRateOfInterest() { return 7; }
}
class AXIS extends Bank {
    @Override int getRateOfInterest() { return 9; }
}
public class Main {
    public static void main(String[] args) {
        Bank[] banks = { new SBI(), new ICICI(), new AXIS() };
        for (Bank b : banks) {
            System.out.print(b.getClass().getSimpleName());
            System.out.println(": %" + b.getRateOfInterest());
        }
    }
}
```

Çıktı (Output)

```
SBI: %8
ICICI: %7
AXIS: %9
```

- Reference tipi **`Bank`**, object tipi **`SBI`/`ICICI`/`AXIS`**.
- Çalışacak method **runtime'da object'in gerçek tipine** göre seçilir: **dynamic method dispatch**.
- Yeni banka eklemek için yalnızca yeni bir subclass yazmak yeterlidir.

@Override Annotation Neden Önemli?

✗ Yazım hatası: sessizce YENİ method

```
class Vehicle {
    void run() { System.out.println("Vehicle"); }
}
class Bike extends Vehicle {
    void Run() {          // büyük R!
        System.out.println("Bike");
    }
}
Vehicle v = new Bike();
v.run();
```

Çıktı (Output)

```
Vehicle
```

- Derlenir ama override **olmamıştır**: logic error.

✓ @Override hatayı derlemede yakalar

```
class Vehicle {
    void run() { System.out.println("Vehicle"); }
}
class Bike extends Vehicle {
    @Override
    void Run() {          // büyük R!
        System.out.println("Bike");
    }
}
```

Derleme Hatası

```
error: method does not override or
implement a method from a supertype
```

- Her override'da **`@Override`** yazmayı alışkanlık edinin.

Access Modifier Kısıtlanamaz

✗ protected → default (daha kısıtlı)

```
class A {
    protected void msg() {
        System.out.println("Hello java");
    }
}
class B extends A {
    @Override
    void msg() {                // HATA
        System.out.println("Hello B");
    }
}
```

Derleme Hatası

```
error: msg() in B cannot override
msg() in A; weaker access privileges
```

✓ protected → public (daha geniş)

```
class A {
    protected void msg() {
        System.out.println("Hello java");
    }
}
class B extends A {
    @Override
    public void msg() {
        System.out.println("Hello B");
    }
}
new B().msg();
```

Çıktı (Output)

```
Hello B
```

static Method'lar Override Edilmez: Method Hiding

```java
class Parent {
    static void hello() { System.out.println("Parent static"); }
    void hi()           { System.out.println("Parent instance"); }
}
class Child extends Parent {
    static void hello() { System.out.println("Child static"); }
    @Override
    void hi()           { System.out.println("Child instance"); }
}
public class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        p.hello();    // reference tipine bakılır (compile time)
        p.hi();       // object tipine bakılır (runtime)
    }
}
```

Çıktı (Output)

```
Parent static
Child instance
```

- static method aynı imzayla subclass'ta yazılırsa parent'ınkini **gizler** (method hiding).
- static çağrı **reference tipine** göre compile time'da çözülür; polymorphism yoktur.
- Bu yüzden `main()` de override edilemez.

Method Overloading vs Method Overriding

Kriter	Overloading	Overriding
Nerede?	Aynı class içinde (veya miras alınanla)	Parent–child class'lar arasında (IS-A)
Parametreler	Farklı olmalı	Aynı olmalı
Return type	Serbest (tek başına yetmez)	Aynı veya covariant
Access modifier	Serbest	Daha kısıtlayıcı olamaz
Polymorphism	Compile-time (static binding)	Runtime (dynamic binding)
static method	Overload edilebilir	Override edilemez (hiding)
private / final	Overload edilebilir	Override edilemez
Exception	Serbest	Daha geniş checked exception fırlatamaz

Bölüm 5



super Keyword

Parent class'a erişmek

super Keyword'ün 3 Kullanımı



super.field

- Parent'ın field'ına erişir
- Child aynı isimde field tanımlayıp parent'ınkini **gizlediyse** kullanılır



super.method()

- Parent'ın method'unu çağırır
- Override edilmiş method'un **parent versiyonu** gerektiğinde
- Davranışı tamamen değiştirmeden **genişletmek** için



super(...)

- Parent'ın constructor'ını çağırır
- Constructor'ın **ilk satırı** olmalı
- Yazılmazsa compiler `super()` ekler

super ile Field ve Method'a Erişim

```java
class Animal {
    String color = "white";
    void eat() { System.out.println("eating..."); }
}
class Dog extends Animal {
    String color = "black";  // parent'ınkini gizler
    @Override
    void eat() { System.out.println("eating bread..."); }
    void show() {
        System.out.println(color);  // Dog'un
        System.out.println(super.color);  // Animal'ın
        eat();  // Dog'un eat()'i
        super.eat();  // Animal'ın eat()'i
    }
}
public class Main {
    public static void main(String[] args) {
        new Dog().show();
    }
}
```

Çıktı (Output)

```
black
white
eating bread...
eating...
```

- Öncelik her zaman en yakındaki (child) üyededir.
- **`super.`** önekiyle parent versiyonuna ulaşılır.
- Parent ile aynı isimde field tanımlamak (field hiding) kafa karıştırır; önerilmez.

super() Açıkça veya Örtük Olarak Çağrılır

Açıkça super()

```
class Animal {
    Animal() { System.out.println("Animal()"); }
}
class Dog extends Animal {
    Dog() {
        super();    // parent constructor
        System.out.println("Dog()");
    }
}
new Dog();
```

Çıktı (Output)

```
Animal()
Dog()
```

super() yazılmasa da compiler ekler

```
class Animal {
    Animal() { System.out.println("Animal()"); }
}
class Dog extends Animal {
    Dog() {
        // super(); otomatik
        System.out.println("Dog()");
    }
}
new Dog();
```

Çıktı (Output)

```
Animal()
Dog()
```

Constructor Çalışma Sırası: Yukarıdan Aşağıya

```java
class A {
    A() { System.out.println("A constructor"); }
}
class B extends A {
    B() { System.out.println("B constructor"); }
}
class C extends B {
    C() { System.out.println("C constructor"); }
}
public class Main {
    public static void main(String[] args) {
        new C();        // tek bir object oluşur
    }
}
```

Çıktı (Output)

```
A constructor
B constructor
C constructor
```

- **`new C()`** → C'nin constructor'ı önce **`super()`** ile B'yi, B de A'yı çağırır.
- Parent kısım **önce** hazırlanır, child kısım sonra.
- Bellekte **tek bir** C object'i oluşur; içinde A ve B'den gelen field'lar da bulunur.

Gerçek Kullanım: super(...) ve super.method()

Person.java / Emp.java

```
class Person {
    int id;
    String name;
    Person(int id, String name) {
        this.id = id;
        this.name = name;
    }
    void display() { System.out.print(id + " " + name); }
}
class Emp extends Person {
    float salary;
    Emp(int id, String name, float salary) {
        super(id, name);                // parent'ın constructor'ı
        this.salary = salary;
    }
    @Override
    void display() {
        super.display();                // parent'ın işini yap
        System.out.println(" " + salary);    // üstüne ekle
    }
}
```

main() çıktısı

```
1 ankit 45000.0
```

- main: **`new Emp(1, "ankit", 45000f).display();`**
- **`super(id, name)`** parent'ın başlatma kodunu tekrar kullanır.
- **`super.display()`** ile davranış **genişletilir**, baştan yazılmaz.

Sık Hata: Parent'ta No-Arg Constructor Yoksa

✗ Örtük super() eşleşmez

```
class Person {
    String name;
    Person(String name) {
        this.name = name;
    }
}
class Emp extends Person {
    Emp() {   // örtük super() -> HATA
    }
}
```

Derleme Hatası

```
error: constructor Person in class Person
cannot be applied to given types
```

✓ Uygun super(...) çağrısı

```
class Person {
    String name;
    Person(String name) {
        this.name = name;
    }
}
class Emp extends Person {
    Emp(String name) {
        super(name);
    }
}
System.out.println(new Emp("Ali").name);
```

Çıktı (Output)

```
Ali
```

- Person'da parametresiz constructor yok.

this ve super Karşılaştırması

Kullanım	this	super
Neyi gösterir?	Mevcut object'i (kendi class'ı)	Mevcut object'in parent'tan gelen kısmını
Field erişimi	this.name	super.name
Method çağrısı	this.display()	super.display()
Constructor çağrısı	this(...) → aynı class	super(...) → parent class
Konum kuralı	Constructor'ın ilk satırı	Constructor'ın ilk satırı
Birlikte kullanım	this() ve super() aynı constructor'da birlikte olamaz	
static context	Kullanılamaz	Kullanılamaz

Alıştırma: Çıktı Nedir?

1

Soru 1

```
class K { static int n; K() { n++; } }
new K(); new K(); new K();
System.out.println(K.n);
```

2

Soru 2

```
class P {
  void f(int x)  { System.out.print("I"); }
  void f(long x) { System.out.print("L"); }
}
new P().f(5);
new P().f(5L);
```

3

Soru 3

```
class A { int v = 1; }
class B extends A { int v = 2; }
A x = new B();
System.out.println(x.v);
```

4

Soru 4

```
class A {
    A() { System.out.print("1"); }
}
class B extends A {
    B() { System.out.print("2"); }
}
class C extends B { }
new C();
```

Özet

static
Class'a ait tek kopya; object'siz erişim; static method instance üyelere doğrudan erişemez; override edilmez.

this
Mevcut object; field/parametre ayrımı, this() ile constructor chaining, return this ile method chaining.

Overloading / Overriding
Overloading: aynı isim, farklı parametre, compile time. Overriding: aynı imza, subclass'ta, runtime. @Override kullanın.

super
Parent'ın field, method ve constructor'ı; super() ilk satır ve yazılmazsa compiler ekler.

Kaynak: tpointtech.com Java eğitim sayfaları (içerik genişletilmiş, örnekler eklenmiş ve hatalar düzeltilmiştir).